

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»**

Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці

ДО ЗАХИСТУ ДОПУЩЕНО

В.о. завідувача кафедри

_____ Олександр КОВАЛЬ

« ____ » _____ 2025 р.

Дипломна робота

**на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
інтелектуальних кібер-фізичних систем в енергетиці»
спеціальності 121 Інженерія програмного забезпечення
на тему: «Підсистема інтелектуального аналізу образів і розпізнавання
об'єктів на земній поверхні»**

Виконав

студент IV курсу, групи ТВ-12

Машталер Владислав Дмитрович

(прізвище, ім'я, по батькові)

_____ (підпис)

Керівник:

ст.викладач Оленєва Ксенія Миколаївна

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Рецензент:

професор кафедри ЦТЕ,

д.т.н., професор Отрох Сергій Іванович

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень із праць інших авторів
без відповідних посилань.

Студент _____

(підпис)

Київ – 2025

**Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»**

Навчально-науковий інститут атомної та теплової енергетики

Кафедра інженерії програмного забезпечення в енергетиці

Рівень вищої освіти перший (бакалаврський)

Спеціальність 121 Інженерія програмного забезпечення

Освітньо-професійна програма «Інженерія програмного забезпечення
інтелектуальних кібер-фізичних систем в енергетиці»

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри

_____ Олександр КОВАЛЬ

«___» _____ 2025р.

ЗАВДАННЯ

на дипломну роботу студенту

_____ Машталеру Владиславу Дмитровичу

(прізвище, ім'я, по батькові)

1. Тема роботи: Підсистема інтелектуального аналізу образів і розпізнавання
об'єктів на земній поверхні

керівник роботи Оленєва Ксенія Миколаївна, старший викладач

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «02» червня 2025р. №1875-с

2. Строк подання студентом роботи «09» червня 2025р.

3. Вихідні дані до роботи: мова програмування Python, середовище розробки
PyCharm

4. Зміст (дипломної роботи) пояснювальної записки (перелік завдань, які потрібно розробити): проаналізувати існуючі системи; розробити підсистему для аналізу
образів і класифікації об'єктів на земній поверхні

5. Перелік ілюстративного матеріалу: приклади використання програмного
застосунку, приклади роботи аналогів, діаграми програмного застосунку

6. Дата видачі завдання «30» жовтня 2024 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів виконання дипломної роботи	Строки виконання етапів роботи	Примітка
1	Отримання завдання	30.10.2024	
2	Дослідження предметної області	31.10.2024 – 01.11.2024	
3	Дослідження існуючих рішень	02.11.2024 – 01.12.2024	
4	Постановка вимог до проєктування системи	02.12.2024 – 12.01.2025	
5	Розробка програмного продукту	13.01.2025 – 11.05.2025	
6	Тестування	12.05.2025 – 15.05.2025	
7	Захист програмного продукту	12.05.2025 – 15.05.2025	
8	Оформлення дипломної роботи	19.05.2025 – 04.06.2025	
9	Передзахист	02.06.2025 – 06.06.2025	
10	Захист	16.06.2025 – 27.06.2025	

Студент

(підпис)

Владислав МАШТАЛЄР

(ім'я, прізвище)

Керівник роботи

(підпис)

Ксенія ОЛЕНЄВА

(ім'я, прізвище)

РЕФЕРАТ

Структура та обсяг дипломної роботи. Робота містить 67 сторінок, 3 рисунки, 2 додатки.

Актуальність теми. У сучасних умовах зростаючого рівня напруги в сфері безпеки критично важливо створити автоматизовану інформаційну систему моніторингу територій, здатну оперативно обробляти великі обсяги гетерогенних даних з різних джерел, виконувати високоточну двоступеневу класифікацію відеокadrів та забезпечувати достовірне розпізнавання типу повітряних суден на території аеропорту навіть за складних умов зйомки.

Метою роботи є розробка інформаційної підсистеми для аналізу супутникових знімків, яка працює в два етапи: фільтрація кадрів за наявністю чи відсутністю літаків, детекція знайдених об'єктів й класифікація на військові або цивільні.

Об'єктом дослідження є процес розробки програмної підсистеми для аналізу образів та розпізнавання об'єктів на земній поверхні.

Предметом дослідження є архітектури, моделі та методи реалізації нейронних мереж для аналізу образів та класифікації об'єктів на земній поверхні.

Методи дослідження включають аналіз існуючих рішень, ознайомлення з ключовими технологіями, проектування та розробка відповідних архітектур.

Практичне значення одержаних результатів: тема важлива тим, що вона закладає основи створення розумних систем обробки відеопотоку в реальному часі. Отримані результати стануть платформою для подібних рішень в обороні, моніторингу та автономному спостереженні.

Ключові слова: аналіз образів, локалізація об'єктів, класифікація об'єктів, мова програмування Python.

ABSTRACT

Structure and scope of the thesis. The work contains 67 pages, 3 figures, 2 appendices.

Relevance of the topic. In today's conditions of increasing tension in the security sphere, it is critically important to create an automated information system for monitoring territories that is capable of quickly processing large volumes of heterogeneous data from various sources, perform highly accurate two-stage classification of video frames, and ensure reliable recognition of aircraft types at the airport even under difficult shooting conditions.

The purpose of the work is the development of an information subsystem for analyzing satellite images, which works in two stages: filtering frames by the presence or absence of aircraft, detection of found objects, and classification as military or civilian.

The object of research is the process of developing a software subsystem for image analysis and recognition of objects on the earth's surface.

The subject of the work are architectures, models, and methods for implementing neural networks for image analysis and classification of objects on the earth's surface.

Research methods include analysis of existing solutions, familiarization with key technologies, design and development of appropriate architectures.

The practical value of the topic is important because it lays the foundation for creating intelligent real-time video stream processing systems. The results obtained will become a platform for similar solutions in defense, monitoring, and autonomous surveillance.

Keywords: image analysis, object localization, object classification, Python programming language.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	7
ВСТУП.....	8
1 ПОСТАНОВКА ЗАДАЧІ	10
2 ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ	13
2.1 Проблематика вхідних даних.....	13
2.2 Ландшафт України.....	14
3 ЗАГАЛЬНІ ПОЛОЖЕННЯ	17
3.1 Огляд сучасних методів класифікації образів	17
3.1.1 ResNet.....	18
3.1.2 InceptionNet.....	19
3.1.3 DenseNet.....	20
3.1.4 EfficientNet.....	20
3.1.5 DenseNet vs. ResNet та інші наслідувані підходи	21
3.2 Огляд сучасних методів локалізації об'єктів на зображенні.....	22
3.3 Користувачі.....	26
3.4 Технології.....	27
3.5 Опис функціональної моделі	27
3.5.1 Класифікація образів	28
3.5.2 Детекція об'єктів образів та їх класифікація	30
3.5.3 Аналіз метрик YOLOV8s після тренування.....	32
4 ЗАСОБИ РОЗРОБКИ.....	38
4.1 Вибір мови програмування для реалізації проекту	38
4.2 Середовище розробки PyCharm.....	42
4.3 Вибір фреймворку для тренування нейронних мереж.....	42
4.4 Бібліотека DVC	46
ВИСНОВКИ.....	49
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	51
ДОДАТОК А Лістинг програми	52
ДОДАТОК Б Презентація.....	59

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API	– Application Programming Interface	– інтерфейс програмування застосунків
GAN	– ResNet Generative Adversarial Network	– генеративно-змагальна нейронна мережа
DL	– Deep Learning	– глибоке навчання
GPU	– Graphics Processing Unit	– графічний процесор
CPU	– Central Processing Unit	– центральний процесор
CSV	– Comma-Separated Values	– текстовий табличний формат
EMA	– Exponential Moving Average	– експоненційне ковзне середнє

ВСТУП

У сучасних умовах стрімкого розвитку технологій дистанційного зондування та обробки великих обсягів просторових даних, системи інтелектуального аналізу образів стають незамінними для вирішення завдань моніторингу та керування транспортною інфраструктурою. Особливо актуальним є автоматизоване виявлення й класифікація об'єктів аеропортної інфраструктури: як цивільних, так і військових аеропортів, а також розпізнавання різних типів повітряних суден. Завдяки поєднанню високоточних супутникових знімків, аерофотознімків із дронів та алгоритмів машинного навчання, система здатна не лише локалізувати злітно-посадкові смуги та авіаційні споруди, а й відрізнити військові об'єкти від комерційних.

Ключовими компонентами платформи є модулі попередньої обробки зображень, глибинного аналізу – з використанням нейронних мереж для класифікації та детекції. При цьому система реалізована за допомогою двох послідовних моделей: перша відповідає за бінарну класифікацію наявності повітряного судна в кадрі, а друга – виконує детекцію та розпізнавання виявлених літаків із поділом на військові та цивільні категорії. Система автоматично виділяє території аеропортів, визначає їх приналежність до військової чи цивільної категорії, а також розпізнає типи літаків (пасажирські, бойові).

Такий підхід забезпечує оперативний моніторинг змін у статусі аеродромів, підвищує точність оцінки авіаційних ризиків і сприяє ефективному прийняттю рішень у сферах оборони, безпеки та цивільного авіаційного планування.

Метою цього дослідження є розробка та впровадження високопродуктивної інтегрованої інформаційної системи, що забезпечить автоматизовану обробку супутникових зображень із такими ключовими можливостями:

- 1) бінарна класифікація зображень:

- автоматичне визначення наявності чи відсутності повітряного судна у кожному кадрі;

- використання компактних та ефективних нейронних мереж, оптимізованих для роботи з великими обсягами даних і мінімальними затримками обробки;

2) детекція та локалізація літаків:

- точне виявлення об'єктів, що мають ознаки повітряного судна, із побудовою обмежувальних рамок (bounding boxes) на зображенні;

- застосування методів багаторівневої сегментації для виділення летальних апаратів на фоні складного ландшафту;

3) класифікація типу повітряного судна:

- розподіл виявлених літаків на військові та цивільні категорії з подальшим уточненням підгруп: бойові, пасажирські;

- інтеграція моделей глибинного навчання, натренованих на багатому наборі маркованих даних, що забезпечує високу точність розпізнавання навіть за умов часткового перекриття і низької контрастності.

1 ПОСТАНОВКА ЗАДАЧІ

Метою даного дослідження є створення інтегрованої інформаційної системи для інтелектуального аналізу супутникових зображень, яка в режимі реального часу здійснюватиме двоступеневу обробку: спочатку бінарну класифікацію кадрів за наявністю або відсутністю повітряного судна, а потім детекцію й розпізнавання виявлених літаків із поділом на військові та цивільні. Для цього буде організовано безперервний потік даних із різноманітних джерел — супутників із різними рівнями просторової роздільної здатності та безпілотних літальних апаратів, — що дозволить отримувати деталізовані зображення земної поверхні як у денний, так і в нічний час доби. Перший модуль системи забезпечить корекцію геометричних спотворень, нормалізацію освітленості для уніфікації якості вхідних даних, після чого компактна нейронна мережа здійснить швидку бінарну класифікацію кадрів, відфільтровуючи ті, що не містять повітряних суден.

У разі позитивного результату первинної оцінки система активує другий модуль, який за допомогою методів об'єктної детекції сформує точні обмежувальні рамки навколо літаків, а потім застосує глибоку класифікацію для віднесення кожного виявленого апарата до категорії «військовий» або «цивільний». Цей підхід ґрунтується на заздалегідь натренованих мережах, що навчалися на різноманітних маркованих датасетах, що містять зображення бойових, транспортних, пасажирських і розвідувальних літаків у різних ракурсах та умовах зйомки. Така послідовна структура забезпечує високу точність класифікації навіть у разі часткового перекриття об'єктів, складного ландшафту чи низької контрастності кадру.

Основні вимоги до функціоналу системи інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні включають:

- збір і попередню обробку зображень;
- бінарну класифікацію наявності літаків;
- детекцію та локалізацію об'єктів;

- класифікацію типів літаків;
- візуалізацію результатів.

Збір і попередня обробка зображень. Система повинна приймати супутникові й аерофотознімки з різним просторовим розширенням, коригувати геометрію кадру, нормалізувати освітленість для уніфікації якості даних.

Бінарна класифікація наявності літаків має відбуватись за допомогою реалізації високоефективної нейронної мережі, здатної в режимі реального часу визначати, чи містить кадр повітряне судно, із точністю не нижче 85 % і мінімальним рівнем хибних спрацювань.

Детекція та локалізація об'єктів має на меті застосування методів об'єктної детекції для побудови обмежувальних рамок (bounding boxes) навколо виявлених літаків із точністю позиціонування не гірше 3 пікселів при середньому масштабі знімка.

Класифікація типів літаків – має відбуватись автоматичний розподіл виявлених об'єктів на військові та цивільні категорії з подальшим уточненням підтипів (бойові, транспортні, пасажирські, розвідувальні) із точністю розпізнавання не нижче 85%

Візуалізація результатів має включати динамічне відображення виявлених об'єктів і їх класифікації.

Висновки до розділу 1

У ході постановки задачі було чітко окреслено необхідність створення інтегрованої інформаційної системи для інтелектуального аналізу супутникових і повітряних зображень у реальному часі. Зазначено, що така система має виконувати двоетапну обробку: спершу бінарну класифікацію кадрів за наявністю чи відсутністю літаків, а потім детекцію й розпізнавання знайдених апаратів з розподілом на військові та цивільні.

Було визначено ключові функціональні модулі: збір і попередня обробка різнорідних зображень (корекція геометричних спотворень та нормалізація освітленості), швидка бінарна нейронна класифікація, точна об'єктна детекція для побудови bounding boxes та глибока класифікація типу літаків. Крім того, встановлено вимоги до точності кожного етапу (не менше 85% для класифікацій, позиціонування виявлених об'єктів із похибкою не більше трьох пікселів) та збереження безперебійного потоку даних із супутників різної просторової роздільності й безпілотників у будь-який час доби.

Візуалізація результатів стане заключним штрихом, що забезпечить оперативне відображення інформації про виявлені об'єкти та їхню категоризацію. Отже, сформульовані цілі та функціональні вимоги створюють ґрунт для подальшої побудови архітектури системи: від вибору оптимальних алгоритмів нейронного аналізу до розгортання реального середовища інференсу.

У наступних розділах докладно розглянуто підбір мови програмування та фреймворків для реалізації зазначених модулів, описуватимемо методи навчання й оптимізації нейронних мереж, а також презентацію інтерфейсних рішень для інтеграції всіх компонентів в єдину інформаційну систему.

2 ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ

Система інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні є комплексом взаємопов'язаних підсистем, об'єднаних єдиними правилами й процедурами обробки просторових даних, які функціонують задля реалізації цілісного процесу автоматичного виявлення та класифікації об'єктів на супутникових і аерофотознімках. До її складу входять підсистеми збору та попередньої обробки зображень, модулі бінарної класифікації наявності літаків, механізми детекції й локалізації об'єктів, а також компоненти глибинного розпізнавання для поділу виявлених літаків на військові та цивільні. Усі підсистеми інтегровані через єдину архітектуру.

2.1 Проблематика вхідних даних

У процесі розробки та впровадження систем інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні ключовою проблемою залишається надзвичайно великий обсяг та різноманіття вхідних даних. Супутникові й аерофотознімки надходять у найрізноманітніших спектральних діапазонах і з роздільною здатністю від кількох десятків сантиметрів до метрів, що вимагає потужних засобів попередньої обробки для корекції геометричних та радіометричних спотворень, очищення від атмосферних перешкод і нормалізації освітленості. Одночасно необхідно забезпечити здатність алгоритмів машинного навчання працювати в реальному часі та відповідати строгим вимогам точності: відсутність літаків у кадрі повинна визначатися майже безпомилково, а виявлення та локалізація повітряних суден потребує надзвичайно точних межових рамок, щоб уникнути хибних тривог. Особливий виклик становлять об'єкти невеликого розміру, частково закриті хмарами або антропогенним ландшафтом, а також варіативність форм і забарвлень військових і цивільних літаків, що ускладнює навчання й узагальнення моделей.

Ще однією суттєвою перешкодою є обмеженість та нерівномірність розмітки даних для тренування систем розпізнавання. Надійна класифікація літаків за належністю до військового або цивільного флоту потребує великих анотованих корпусів знімків, у яких зазначено не лише положення об'єкта, а й його тип і підкатегорію. Збір таких даних у достатній кількості та високій якості — дорога й ресурсомістка операція, що часто обмежується доступом до комерційних чи закритих військових знімків. Окрім цього, виклики становить і обчислювальна складність: двоступенева архітектура, що складається з бінарної класифікації наявності літаків і подальшої детекції з розпізнаванням їх приналежності, вимагає продуманої оптимізації інференсу (quantization, pruning) та масштабованої хмарної інфраструктури.

2.2 Ландшафт України

В українських умовах до загальних проблем додаються специфічні вимоги стратегічного та безпекового характеру. Географічне положення країни, що межує з кількома державами та безпосередньо постраждала від воєнних дій, обумовлює потребу в безперервному моніторингу повітряного простору й інфраструктури аеропортів – як цивільних, так і військових. Недостатньо розвинена національна система збору та обміну супутниковими даними, а також обмежена доступність високоякісних знімків у відкритому доступі, створюють додаткові перешкоди для аналітиків і розробників. Водночас наявність в Україні розвиненого ІТ-сектора й зростаючий попит з боку державних відомств та приватних компаній у галузі безпекових технологій відкривають широкі можливості для впровадження інноваційних рішень. Реалізація такої системи дозволить оперативно виявляти та класифікувати літаки в межах національних кордонів, підвищити рівень авіаційної безпеки і забезпечити прозорість повітряного руху для всіх зацікавлених сторін.

Висновки до розділу 2

У другому розділі було детально описано предметну область системи інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні. Показано, що система складається з низки взаємопов'язаних підсистем: від збору та попередньої обробки супутникових і аерофотознімків до модулів бінарної класифікації, об'єктної детекції й глибокої класифікації типів літаків. Кожен з цих компонентів виконує свою ключову функцію: від корекції геометричних та радіометричних спотворень на вході до точного побудування обмежувальних рамок і диференціації між військовими і цивільними апаратами. Завдяки єдиній архітектурі забезпечується безшовна передача результатів між етапами, що гарантує цілісність і послідовність інформаційного потоку.

Водночас було підкреслено складність роботи з вхідними даними: різнорідність джерел (супутники з різною роздільною здатністю, БПЛА, кадри з різних спектральних діапазонів), необхідність корекції атмосферних й геометричних викривлень та нормалізації освітленості, а також гостру потребу в масштабованих алгоритмах машинного навчання, здатних працювати в реальному часі. Особливу увагу приділено проблемі недостатньої та нерівномірної розмітки даних для тренування моделей розпізнавання типів літаків — дефіциту анотованих корпусів, що ускладнює отримання високої точності класифікації. Окремо розглянуто й питання обчислювальної складності двоступеневої архітектури, яке вимагає впровадження оптимізацій (квантизація, зменшення ваг моделей) та застосування масштабованої інфраструктури (хмарні сервіси, GPU-кластери).

На прикладі України було продемонстровано, що вітчизняні реалії накладають власні вимоги: потреба в безперервному моніторингу повітряного простору й аеропортів, обмежений доступ до високоякісних відкритих супутникових даних і водночас наявність потужного ІТ-сектору та державного запиту на безпекові технології. Саме ці умови створюють як виклики (дефіцит розмічених знімків, обмеженість доступних ресурсів), так і можливості (розробка

локальних інноваційних рішень, інтеграція з державною інфраструктурою та приватними ініціативами). Таким чином, детальна характеристика предметної області й специфіки українського ринку стало міцною основою для вибору технічних підходів у наступних розділах, зокрема для обґрунтування вибору мов програмування, фреймворків і алгоритмів, а також визначення подальших етапів реалізації та валідації розробленої системи.

3 ЗАГАЛЬНІ ПОЛОЖЕННЯ

Додаток інформаційної системи інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні створено з метою забезпечення точного та ефективного моніторингу повітряного простору для різних категорій користувачів – від оперативних аналітиків до керівників служб безпеки. Головним завданням цієї системи є автоматична й своєчасна класифікація супутникових зображень на ті, що містять літаки, та ті, що їх не містять, а також детекція виявлених суден з поділом на військові та цивільні. Завдяки цьому оператори отримують актуальну інформацію про розташування літаків, їх маршрути та статус, що значно підвищує швидкість реакції на потенційні загрози й оптимізує прийняття рішень у режимі реального часу.

Метою розробленої системи є своєчасне виявлення та класифікація повітряних суден із високою точністю й надійністю за заданими параметрами роздільної здатності зображень та швидкості обробки. Вона покликана задовольняти вимоги оборонних структур та цивільних авіаційних операторів, забезпечуючи мінімізацію хибних спрацювань і оптимальне розподілення обчислювальних ресурсів, а також гнучке масштабування в умовах зростання обсягів даних.

3.1 Огляд сучасних методів класифікації образів

У сучасних умовах зростання обсягів зображень і відеоматеріалів, а також розвитку апаратних можливостей комп'ютерних систем, завдання автоматичної класифікації образів набуло особливої актуальності. Раніше методи класифікації були здебільшого засновані на ручному виділенні ознак і традиційних алгоритмах машинного навчання, тоді як за останнє десятиліття глибокі нейронні мережі радикально змінили підходи до аналізу візуальної інформації. Однією з найважливіших віх стало запровадження архітектур із залишковими зв'язками

(ResNet), а також мереж, що поєднують різноманітні масштабуючі фільтри (InceptionNet), що дозволяє подолати проблему затухання градієнту та підвищити точність класифікації навіть на надскладних наборах даних. Надалі з'явилися DenseNet із щільними з'єднаннями, EfficientNet із композитним масштабуванням, а також трансформерні підходи (Vision Transformer), які привнесли концепцію уваги до обробки зображень. Цей огляд має на меті систематизувати і проаналізувати основні сучасні архітектури та підходи до класифікації образів, розкрити їхні принципи роботи, переваги й обмеження, а також показати, як розвиток кожного з напрямів вплинув на загальну еволюцію галузі. Далі буде детально розглянуто ResNet, InceptionNet, DenseNet, EfficientNet, MobileNet та інші передові рішення, що наразі визначають стандарти точності й ефективності в задачах класифікації образів.

3.1.1 ResNet

Однією з найважливіших інновацій у сфері глибинного навчання для класифікації зображень стала архітектура ResNet (Residual Network), запропонована Хе й співавт. у 2015 році. Головна ідея ResNet полягає у використанні залишкових (residual) зв'язків, які дозволяють ефективно передавати інформацію через надзвичайно велику кількість шарів без втрати градієнту. Залишковий блок складається з двох чи трьох послідовних згорткових шарів, після яких результат додається до вхідного сигналу блоку (identity shortcut). Такий підхід вирішує проблему «затухання градієнту», що істотно обмежувало глибину традиційних мереж. ResNet-50, ResNet-101 та ResNet-152 мають відповідно 50, 101 і 152 шари, але завдяки залишковим зв'язкам ці глибокі моделі навчаються стабільно. ResNet показав вражаючі результати на ImageNet (1-е місце в змаганні ILSVRC 2015), скоротивши топ-1 і топ-5 помилки порівняно з попередніми підходами. Крім того, архітектура ResNet стала основою для багатьох наступних моделей (наприклад, ResNeXt, Wide ResNet та ін.), які розширили ідейну базу

залишкових блоків, додаючи уніфікацію групових згорток, збільшені канали та інші оптимізації.

3.1.2 InceptionNet

InceptionNet (також відома як GoogLeNet), запропонована Сцуболтом і співавт. у 2014 році, принесла нове уявлення про те, як конструювати згорткові мережі з різноманітними масштабуючими фільтрами. Ключовим у Inception-модулі є паралельне застосування фільтрів розміром 1×1 , 3×3 та 5×5 , а також операцій максимального пулінгу (max pooling) до одного й того самого вхідного тензора. Результати різних фільтрів об'єднуються (concatenation) на виході модуля. Щоб зменшити обчислювальну складність великих згорток (3×3 та 5×5), перед ними ставлять згортки 1×1 , які зменшують розмірність (reduction) каналів. Такий підхід дозволяє моделі навчати фільтри різної рецептивної області одночасно, забезпечуючи широке охоплення просторових патернів. Архітектура GoogLeNet (Inception v1) налічує 22 шари глибини (22 шари з вагами) та показала високу точність на ImageNet 2014 з мінімальною кількістю параметрів (приблизно 6,8 млн), значно менше, ніж у аналогічних моделей того часу. У подальших версіях (Inception v2, v3, v4 та Inception-ResNet) покращено баланс між точністю та швидкістю: модулі оптимізовані за допомогою факторизації великих згорток (наприклад, розбиття 5×5 у послідовність 1×5 та 5×1), а в Inception-ResNet інтегровані залишкові з'єднання для пришвидшення навчання та покращення плинності градієнту. Завдяки гнучкості та відносно невеликій кількості параметрів Inception-сімейство залишається популярним вибором для задач класифікації образів і досі використовується як основа для багатьох прикладних рішень.

3.1.3 DenseNet

DenseNet (Densely Connected Convolutional Networks), запропонована Хуангом і співавторстві у 2017 році, вивела ідею покращення потоку інформації на новий рівень. У DenseNet кожен шар підключений безпосередньо до всіх своїх попередників, тобто вихід кожного шару передається на вхід усім наступним шарам у тому ж блоці. Це означає, що замість звичайного лінійного з'єднання «шар $\rightarrow$ шар», маємо «шар $\rightarrow$ усі подальші шари». Така щільна (dense) когезія забезпечує:

- кращий потік градієнту між шарами;
- меншу дублювання ознак (оскільки унікальні ознаки не «забуваються»);
- потребу у меншій кількості параметрів, бо кожен шар отримує «вже вивчену» інформацію, що дозволяє використовувати відносно вузькі шари.

DenseNet-121, DenseNet-169, DenseNet-201 і DenseNet-264 позначають кількість шарів у моделі. На ImageNet DenseNet показали порівнянну або кращу точність у порівнянні з ResNet та VGG, при цьому маючи менше параметрів та менші обчислювальні витрати. Окрім того, щільні з'єднання полегшують реюз (повторне використання) ознак, що сприяє кращій генералізації на нові набори даних. На основі DenseNet також з'явилися варіанти, які поєднують щільні шари з різними техніками регуляризації та адаптивного навчання, що подальше підвищує точність у задачах класифікації та сегментації.

3.1.4 EfficientNet

EfficientNet, розроблена Танном і співавт. у 2019 році, запропонувала вельми елегантний і математично обґрунтований підхід до масштабування нейронних мереж. Замість довільного збільшення глибини, ширини або роздільної здатності вхідного зображення, EfficientNet використовує «компонентне масштабування» (compound scaling), яке одночасно збільшує всі три аспекти відповідно до

заздалегідь визначених коефіцієнтів. Базовим блоком EfficientNet є модифікований MBConv (Mobile Inverted Bottleneck Convolution) – той же, що використовується в MobileNetV2, але з додатковими оптимізаціями: Squeeze-and-Excitation (SE) блоками для підсилення важливих каналів, а також зорієнтованою на увагу схемою розкриття (expansion). Лінійка моделей EfficientNet позначається як B0, B1, ..., B7, де B0 – це невелика модель (~5,3 млн параметрів), а B7 – максимально велика (~66 млн параметрів). При тестуванні на ImageNet EfficientNet-B7 досяг 84,4 % точності топ-1, при цьому значно перевершуючи інші моделі при порівнянній кількості параметрів. Завдяки збалансованому масштабуванню EfficientNet демонструє високу продуктивність як на ресурсно-обмежених пристроях (EfficientNet-Lite), так і на великих обчислювальних платформах, що робить її універсальним вибором для задач класифікації образів у промислових та мобільних застосунках.

3.1.5 DenseNet vs. ResNet та інші наслідувані підходи

Хоч DenseNet та ResNet мають подібну мету – поліпшити потік градієнту в глибоких мережах – реалізація відрізняється: ResNet додає вхідні ознаки до виходу блока, тоді як DenseNet передає вихід кожного шару як вхід до всіх наступних. Існують також проміжні варіанти, наприклад ResNeXt (2017), який поєднує залишкові з'єднання з груповими згортками (cardinality), забезпечуючи ширшу архітектуру без надмірного збільшення обчислювальної складності. Інший приклад – Wide ResNet, в якому замість глибини збільшується ширина каналів у ResNet-блоках. Ці підходи демонструють, що комбінація ідей залишкових, щільних та групових згорток дає змогу знаходити оптимальний компроміс між швидкістю навчання, якістю класифікації та обчислювальними витратами.

3.2 Огляд сучасних методів локалізації об'єктів на зображенні

У сучасному просторі комп'ютерного зору існує безліч архітектур для локалізації об'єктів, які розвивалися від класичних двоетапних моделей до швидких одноетапних фреймворків, а з останніх років – і до трансформерних і без'якірних підходів.

3.2.1 Faster R-CNN (двоетапний, регіон-пропозиційний підхід)

Уже кілька років Faster R-CNN лишається стандартом для високоточних систем локалізації завдяки двоетапному порядку обробки: спершу мережа генерує регіонні пропозиції (Region Proposal Network, RPN), а потім у другому етапі витягнуті ознаки в кожному запропонованому регіоні класифікуються й уточнюються координати bounding-box. У першому блоці RPN на основі згорток створює тисячі потенційних ящиків різного розміру та співвідношення сторін, після чого фільтрує їх за допомогою короткого класифікатора та регресора. Другий етап використовує ці пропозиції для деталізованого прогнозу: застосовується ROI Pooling (або ROI Align у пізніших версіях) до витягнення фічів і подальші повнозв'язні (FC) шари для остаточної класифікації та регресії коригувань. Такий підхід дає високу точність, особливо в умовах складних сцен та дрібних об'єктів, проте вимагає значних обчислювальних ресурсів і працює у більшій кількості мілісекунд на зображення, ніж швидкі одноетапні моделі. Улучшення на зразок Cascade R-CNN та R-CNN із Feature Pyramid Network (FPN) значно підвищили якість, але основна концепція двоетапної схеми залишилася стабільною: суворе відділення етапу пропозицій від етапу класифікації дозволяє зосередити обчислювальну потужність на найобіцяльніших регіонах, що найкраще працює в точних системах аналітики, медичної візуалізації та автономних автомобілях, де затримки менш критичні, ніж проникновність та деталізація.

3.2.2 SSD (Single Shot MultiBox Detector)

SSD започаткував нову еру одноетапних детекторів завдяки ідеї передбачати категорії та координати bounding-box із кінцевих карт ознак на різних масштабах без оглядального етапу пропозицій. Замість генерування тисячі точних пропозицій SSD розбиває задачу на кілька рівнів з різними роздільними здатностями фіч-карт: великі карти (наприклад, 38×38) відповідають за виявлення дрібних об'єктів, а менші (1×1) – за великі об'єкти. На кожному рівні фіксовано декілька “якорів” (anchor boxes) із різними співвідношеннями сторін, і для кожного такого “якоря” мережа напряму прогнозує ймовірності кожного класу та зміщення від центру. Така архітектура дозволяє SSD обробляти відеопотік майже в реальному часі (близько 50 FPS на GPU), а одночасне використання багатовимірних фіч-карт підвищує стійкість до різномасштабних об'єктів. Незважаючи на те, що SSD початково поступався Faster R-CNN у точності, подальші видання (SSD512, DSSD) і удосконалення структурної частини Back-bone (наприклад, використання ResNet або MobileNet як основи) значно закрили цей розрив. SSD залишається популярним вибором для застосувань, де потрібно збалансувати швидкість і точність, зокрема для вбудованих пристроїв, мобільних застосунків і базових відеоаналітичних систем.

3.2.3 YOLO (You Only Look Once) та його еволюція

Одноетапний фреймворк YOLO зробив революцію в сфері детекції ще із виходом першої версії, запропонувавши ділити зображення на сітку $S \times S$ і для кожної клітинки прогнозувати фіксовану кількість прямокутників із координатами та ймовірностями класів за один прогін мережі. Найважливіші впровадження лежать у архітектурі, оптимізованій для високої швидкості: спрощена Backbone (тематика Darknet у ранніх версіях), агрегація ознак шляхом маршрутної передачі (skip connections) та одночасне прогнозування декількох «якірних» штук. З версії

YOLOv3 було введено три рівні детекції з різною роздільною здатністю фіч-карт, що значно покращило роботу з малими об'єктами. Далі YOLOv4 та YOLOv5 додали bag of freebies і bag of specials – різні прийоми, такі як mosaic augmentation, cutmix, різні функції втрат, нові блоки CSP (Cross Stage Partial), SPP (Spatial Pyramid Pooling) та PANet (Path Aggregation Network) для кращої агрегації багатомасштабних ознак. YOLOv6 і YOLOv7 продовжили оптимізувати збалансування між швидкістю та точністю, а в YOLOv8, що вийшов наприкінці 2024 року, з'явилася нативна підтримка PyTorch-інтерфейсу, усунені жорсткі «якірні» обмеження за рахунок anchor-free принципу (переходу до токенів «point-based»), покращена робота з аліасами класів і потужніший модуль розгортання ONNX для мобільних пристроїв. Завдяки своїй легкій інтеграції та високій швидкості (100+ FPS на сучасних GPU) сімейство YOLO широко застосовується в реальному часі: відеоспостереженні, автономній навігації, дронах та IoT-системах, коли критично важливо отримати відповідь негайно.

3.2.4 EfficientDet (компроміс між швидкістю та точністю через NAS та BiFPN)

EfficientDet поєднав ідеї автоматичного пошуку архітектури (Neural Architecture Search, NAS) із оптимізованими багатомасштабними злиттями ознак (Bi-directional Feature Pyramid Network, BiFPN) та компактними backbone EfficientNet. У центрі EfficientDet лежить унікальний підхід до масштабування всіх компонентів одночасно – Backbone, FPN і голови детектора — за допомогою єдиної множникової змінної ϕ . Це дозволяє створювати сімейство моделей від EfficientDet-D0 (дуже легка, призначена для мобільних платформ) до D7 (дуже точна, для серверних застосунків). BiFPN реалізує зворотній та прямий потоки інформації між рівнями фіч-карт, забезпечуючи ефективну агрегацію тимчасових та просторових ознак із мінімальними витратами. Крім того, EfficientDet активно використовує прості, але дієві трюки оптимізації: depthwise-separable convolutions,

weighted feature fusion, термін «compound scaling» для одночасного зміщення глибини, ширини й роздільної здатності. На стандартних бенчмарках COCO EfficientDet-D3–D5 показують співвідношення mAP/галочка-обчислювальних ресурсів, значно краще за Faster R-CNN і SSD, з практично лінійним зростанням продуктивності зі збільшенням ф. Використання EfficientDet особливо виправдане у проєктах, де важливо отримати високу точність, але зберегти обмежені обчислювальні ресурси (наприклад, edge-комп'ютинг або дрони з обмеженим енергоспоживанням).

3.2.5 CenterNet (ключово-точковий, без'якірний підхід)

CenterNet запропонував новий погляд на локалізацію, оминаючи необхідність у «якірних» ящиках: модель прогнозує теплові карти центрів об'єктів (object centers), а також для кожного виявленого центру робить регресію до чотирьох крайніх координат (або до розміру ковпачка bounding-box). Ключова інновація полягає в тому, що у фазі навчання досить маркувати лише центр та розміри об'єкта, що значно спрощує анотацію. Замість складного підбору та фільтрації anchors CenterNet використовує 2D-heatmap, у якій вищі значення відповідають ймовірним центрам об'єктів. Для кожного виявленого піку на карті (вершини, що включають пару координат) проводиться регресія до лівої, правої, верхньої й нижньої меж. Такий підхід дуже добре працює зі щільно розташованими об'єктами, оскільки shift-invariant heatmap дозволяє моделі вирізняти тісно розташовані об'єкти без конфліктів між розмірними ящиками. Недоліком є дещо гірша продуктивність із дуже дрібними об'єктами й підвищена чутливість до вибору порогу для пікових значень heatmap. Проте у версіях CenterNet (наприклад, DLA-34 або HourglassNet Backbone) досягнуто суттєвого поліпшення якості, а за рахунок простої архітектури цей метод легко модифікується для багатозадачного виявлення (детекція + семантичні уточнення), особливо у відеоаналітиці.

3.3 Користувачі

Користувачами системи інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні є фахівці з оборонних та безпекових структур, аналітики розвідувальних підрозділів, оператори повітряного контролю та стратегічного планування, а також представники цивільних авіаційних служб і екстрених оперативних центрів. Інтерфейс платформи створено з урахуванням потреб військових аналітиків: завдяки картографічному відображенню виявлених літаків користувачі можуть не лише бачити їхнє поточне розташування, а й відстежувати траєкторії руху впродовж обраного інтервалу часу, зі спеціальними позначками для бойових і транспортних літальних апаратів. У разі виявлення невідомих або потенційно небезпечних об'єктів система автоматично формує детальний аналітичний звіт із координатами, часом появи та класифікацією судна, надсилаючи тривожне сповіщення до командних центрів і забезпечуючи можливість оперативного реагування.

Окрім військової сфери, систему використовують служби цивільної авіації для моніторингу загального трафіку та запобігання порушенням повітряних кордонів, а також підрозділи з надзвичайних ситуацій для координації гелікоптерних рятувальних операцій. Аналітики отримують доступ до статистики за детектованими об'єктами, включно з кількістю прольотів над критично важливими об'єктами інфраструктури, оцінкою навантаження на військові аеродроми й прогнозом можливих сценаріїв руху сил противника. У інтерфейсі передбачено гнучкі фільтри для відбору даних за типом повітряного судна, часовим інтервалом і регіоном, а також можливість експорту звітів в стандартизованих форматах для подальшого аналізу й інтеграції з іншими оборонними інформаційними системами.

3.4 Технології

Програмна система інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні реалізована мовою Python, що забезпечує гнучкість розробки та легкість інтеграції з численними бібліотеками для обробки зображень і роботи з даними. Основою для побудови нейронних мереж служить фреймворк PyTorch, який дозволяє швидко експериментувати з архітектурами й оптимізувати процес навчання завдяки динамічному графу обчислень та широкому набору інструментів для прискорення інференсу.

Для вирішення завдання бінарної класифікації наявності літаків у кадрі застосовується модель ResNet з глибиною 50 шарів (ResNet-50), натренована на великому наборі маркованих супутникових знімків. ResNet-архітектура зі своєю властивістю «залишкових» (residual) зв'язків дозволяє ефективно навчати глибокі мережі без проблеми згасання градієнта, що гарантує високу точність розпізнавання навіть за складних умов освітлення та фонового шуму.

Для детекції та локалізації літаків у просторі використовується алгоритм YOLO (You Only Look Once), адаптований до специфіки супутникових знімків із різною роздільною здатністю. Цей підхід дозволяє здійснювати одноетапну детекцію об'єктів із високою швидкістю (до десятків кадрів на секунду) та точністю позиціонування рамок навколо літаків. Комбінація ResNet для класифікації та YOLO для детекції створює єдиний конвеєр обробки зображень, що забезпечує надійне й оперативне визначення типу та розташування повітряних суден на земній поверхні.

3.5 Опис функціональної моделі

Інформаційна система інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні – це програмний комплекс для автоматизованої обробки супутникових та аерофотознімків із метою виявлення та класифікації

повітряних суден. До її складу входять модулі імпорту знімків із різних джерел, попередньої обробки – корекції геометрії, нормалізації освітленості та усунення шумів, – двоступеневої обробки зображень (спочатку бінарна класифікація наявності літаків, потім детекція з побудовою обмежувальних рамок і глибинна класифікація «військовий/цивільний»), а також засоби візуалізації отриманих результатів.

Основна функція системи полягає в автоматичному моніторингу наземного простору в режимі реального часу. Після надходження зображення з супутника або безпілотної модуль попередньої обробки готує кадр для аналізу, далі нейронна мережа на основі ResNet здійснює первинну класифікацію «літак / не літак», а алгоритм YOLO детектує об'єкти, формуючи bounding boxes навколо кожного судна. Кожний виявлений об'єкт проходить остаточну класифікацію за належністю до військової чи цивільної категорії, після чого дані оновлюються в центральному сховищі. Всі результати зберігаються для подальшого аналізу і звітування.

3.5.1 Класифікація образів

Процес навчання моделі ResNet для бінарної класифікації кадрів за наявністю літаків починається зі створення та попередньої обробки тренувального датасету, який складається з тисяч супутникових і аерофотознімків (рисунки 3.1), розмічених як «літак є» або «літак відсутній». Спочатку кожне зображення проходить етап корекції геометричних спотворень та нормалізації яскравості, після чого застосовуються методи аугментації: випадкові обертання, відбиття по горизонталі та зміна масштабу. Це дозволяє моделі навчитися розпізнавати літаки в різних орієнтаціях та умовах освітлення.

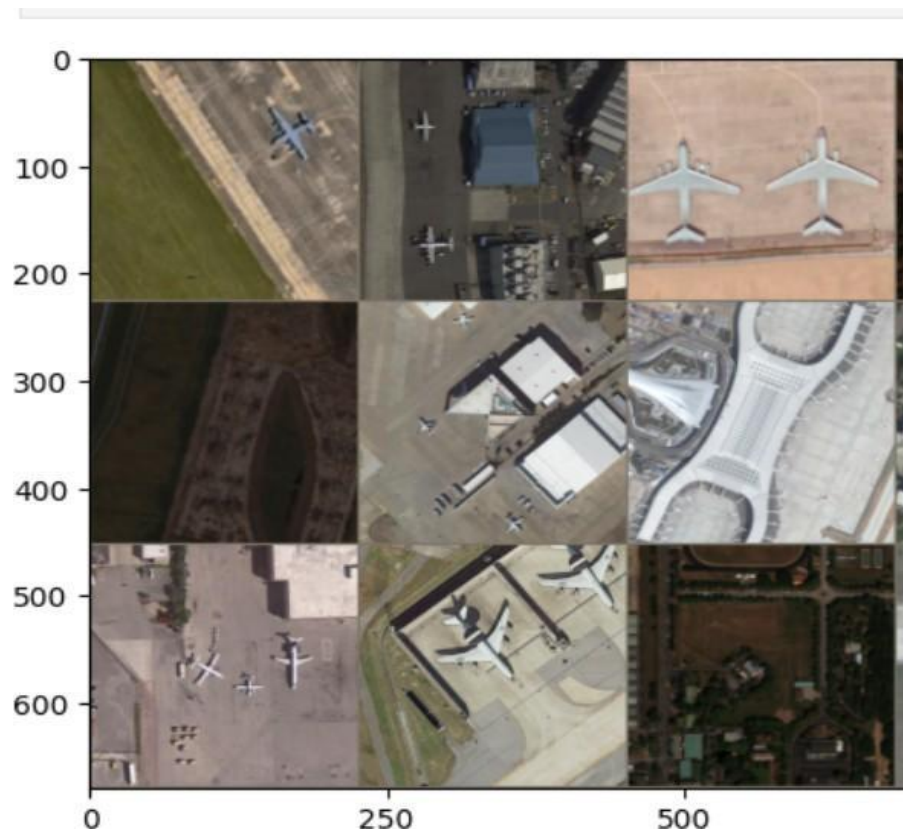


Рисунок 3.1 - Приклад зображень, що використовувалися під час тренування моделі

Архітектура ResNet-34, обрана для класифікації, відкриває можливості побудови глибокої нейронної мережі з залишковими зв'язками, які запобігають згасанню градієнта під час зворотного поширення помилки. На етапі ініціалізації ваги мережі завантажуються з попередньо натренованої моделі на ImageNet, після чого виконується тонка донавчання (fine-tuning) на нашому спеціалізованому датасеті супутникових знімків. Спочатку заморожуються всі шари, окрім останнього блоку, щоб швидко адаптуватися до нової предметної області, а в наступних ітераціях поступово «розморожуються» попередні шари для більш глибокої оптимізації.

Оптимізація проводиться з використанням стохастичного градієнтного спуску зі змінним кроком навчання (learning rate scheduler), який починає з 0.001 і зменшує його вдвічі кожні 10 епох. Під час навчання контролюється метрика точності (accuracy) та функція втрат (binary cross-entropy), щоб забезпечити високу

чутливість (recall) при мінімальному рівні хибних спрацьовувань (false positives).

Після досягнення стабільного зростання показників на валідаційному наборі проводиться остаточне оцінювання на відкладеному тестовому наборі, де модель демонструє точність не менше 85 %. Приклад метрик моделі продемонстровано на рисунку 3.2.

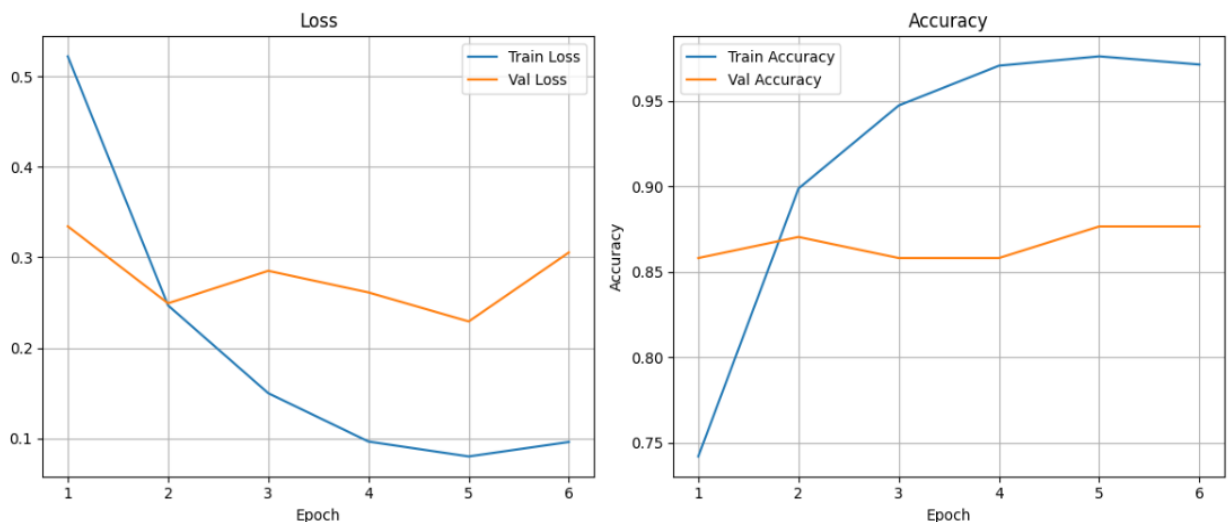


Рисунок 3.2 - Приклад метрик моделі

Таким чином, послідовна процедура попередньої обробки даних, тонкої настройки ResNet із використанням стратегій заморожування шарів та адаптивного навчання, а також суворий контроль якості навчання забезпечують надійний фундамент для наступних етапів системи – детекції та класифікації повітряних суден за категоріями «військові» й «цивільні».

3.5.2 Детекція об'єктів образів та їх класифікація

У рамках другого етапу обробки супутникових та аерофотознімків система переходить від бінарного розпізнавання наявності літаків до їх точної детекції та класифікації в межах одного конвеєра на основі архітектури YOLO (You Only Look Once). Першим кроком є підготовка датасету: кожен кадр із позначеними рамками (bounding boxes) навколо усіх наявних повітряних суден доповнюється

інформацією про їх приналежність до категорії «військові» або «цивільні». Для підвищення стійкості моделі до змін масштабу та ракурсів зображень застосовують аугментацію – випадкові зсуви, масштабування та корекцію колірного балансу.

Архітектура YOLO v5 обрана за її здатність поєднувати високу швидкість інференсу із збереженням точності позиціювання рамок. Модель навчається одночасно розпізнавати місцезнаходження кожного об'єкта та одразу класифікувати його клас – «військовий» або «цивільний» літак. Під час навчання основну увагу приділено балансуванню ваг втрат: локалізаційна складова відповідає за помилку зміщення рамки, а компонент класифікації — за коректність присвоєння категорії.

Процедура навчання починається з використання попередньо натренованих ваг на загальних наборах даних (COCO, ImageNet), що дає змогу модельній архітектурі засвоїти базові уявлення про об'єкти. Далі виконується донавчання на нашому спеціалізованому датасеті з супутникових знімків: із поступовим зменшенням learning rate від початкового 0.01 до 0.001 та застосуванням стратегії warm-up для стабілізації градієнтів у перші епохи. З метою уникнення перенавчання використовують техніки regularization, зокрема DropBlock та випадкову деформацію сітки (mosaic augmentations).

Після завершення тренування модель тестується на окремому наборі валідаційних знімків із різними умовами освітлення та ландшафтним фоном. Оцінювання здійснюється за метриками mAP50 для локалізації та precision-recall для кожної категорії. Отримані результати демонструють високу точність виявлення (mAP понад 80 %) та стабільне розрізнення військових і цивільних літаків навіть у складних умовах, коли об'єкти частково перекриті або слабо контрастні.

Застосування моделі YOLO у складі системи інтелектуального аналізу зображень забезпечує одноетапну, але водночас надійну та швидку детекцію й класифікацію повітряних суден. Це дає змогу в режимі реального часу відстежувати кілька сотень кадрів на секунду, миттєво оновлювати карту

розташування літаків і одразу повідомляти користувачів про потенційно небезпечні або непізнані об'єкти у повітряному просторі.

3.5.3 Аналіз метрик YOLOV8s після тренування

Нижче наведено детальний аналіз отриманих метрик під час тренування моделі YOLOv8s. На рисунку 3.3 відображено десять графіків (по п'ять зверху й знизу), що показують динаміку трьох видів втрат (loss) на тренувальній і валідаційній вибірках, а також чотирьох основних метрик якості (precision, recall, mAP@50 і mAP@50–95).

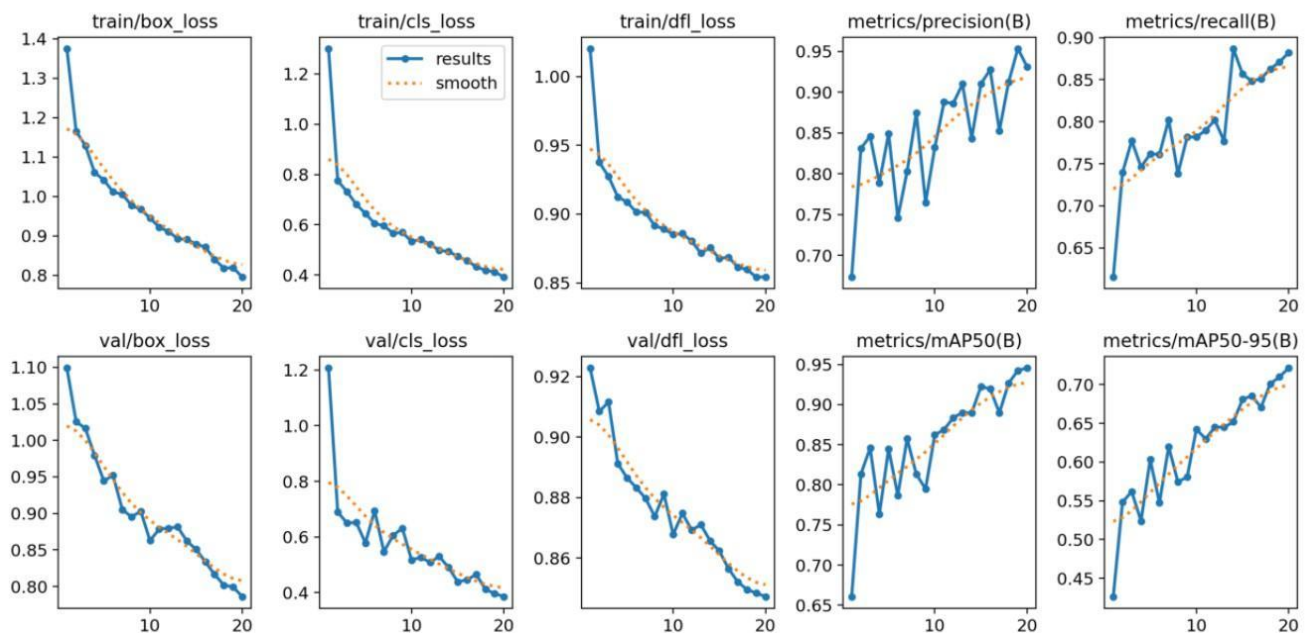


Рисунок 3.3 – Динаміка трьох видів втрат

Розглянемо представлені графіки детальніше.

Динаміка трьох компонент втрати (loss):

- **train/box_loss** (перший графік згори зліва). Початкове значення $\approx 1.4 - 1.35$ на 1-2-ій епохах, що свідчить про те, що спочатку модель дуже невпевнено відображає «границі» літаків (bounding boxes). Протягом 20 епох **box_loss** стабільно зменшується до кінцевого значення ≈ 0.8 ;

- відсутні різкі стрибки чи затримки в зниженні, отже, навчання «рамки» (тобто місця, де модель передбачає літак) відбувається досить плавно і без проблем із нестабільністю градієнту.

train/cls_loss (другий графік згори):

- початкове значення $\approx 1.25 - 1.2$, що означає: на початку навчання мережа надто часто помилково відносить піксель або пропозицію об'єкта до невірної категорії (цивільний vs військовий);

- до кінця 20-ї епохи cls_loss опускається приблизно до $0.75 - 0.8$;

- зниження cls_loss показує, що мережа поступово «вчиться» відрізняти цивільні літаки від військових і фонового простору. Повільне, але стабільне згладжування графіка (за пунктиром «smooth») свідчить про відсутність суттєвих коливань у процесі оптимізації.

train/dfl_loss (третій графік згори праворуч):

- dfl_loss (Distance-to-First-Line loss) оцінює, наскільки точно модель позиціонує межі рамки об'єкта. Початкове значення ≈ 1.05 на першій епосі;

- до 20-ї епохи dfl_loss зменшується приблизно до 0.85 ;

- це зниження означає, що мережа поступово вдосконалює координати «bounding box», отже, локалізація літаків стає дедалі точнішою.

val/box_loss (перший графік знизу зліва):

- початковий тренд: $\approx 1.1 - 1.05$ на першій епосі, потім плавне зниження до $\approx 0.85 - 0.9$ до середини траєкторії (епохи 10 – 12) і далі близько $\approx 0.8 - 0.85$ до 20-ї епохи;

- певні незначні флуктуації в діапазоні $0.9 - 0.85$ між 5 – 10 епохами, але загалом тренд вниз чіткий та налагоджений. Це вказує на те, що модель не «переобучається», а адекватно узагальнює здобуті знання на валідаційних даних.

val/cls_loss (другий графік знизу):

- стартує біля ≈ 1.2 , потім досить різко опускається до ≈ 0.7 біля 6 – 7 епохи. Після цього – невеликі коливання у діапазоні $0.7 - 0.9$ до кінця навчання, але

загальна тенденція також рухається вниз, завершуючись на $\approx 0.6 - 0.65$;

- коливання можуть свідчити про деяку складність у розрізненні класів «цивільний»/«військовий» у валідаційному наборі (можливо, через приклади, де літак частково закритий або погано освітлений). Однак кінцевий рівень `cls_loss` – достатньо низький і наближається до тренувального, що означає хорошу здатність узагальнювати.

`val/df_l_loss` (третій графік знизу праворуч):

- на початку ≈ 0.92 , далі плавний спад до $\approx 0.87 - 0.88$ до середини траєкторії (епохи 6 – 8), після чого – невелике короткострокове зростання близько 7 – 9 епох (можливо, модель «пробує» нові комбінації ваг), але з 10-ї епохи тренд назад вниз і завершення близько ≈ 0.85 на 20-й;

- така картина – дозволяє стверджувати, що валідаційна «точність» локалізації не відстає суттєво від тренувальної, отже, не відбувається помітного переобучення локалізаційного підмодуля.

Динаміку основних метрик якості можна прослідкувати наступним чином: на правій частині кожного ряду розташовані графіки чотирьох важливих метрик. Позначка «(B)» умовно означає «загальна метрика по всіх об'єктах (batch)» чи по класу «B» (bird/background) у прикладах YOLO; у нашому випадку це метрики відносно усіх передбачень (усіх класів літаків).

`metrics/precision (B)`:

- початкове значення `precision` ≈ 0.75 (приблизно на 1 – 2-й епосах);
- помітні коливання (до $\sim 0.65 - 0.9$) у діапазоні епох 2 – 6 із подальшим плавним підйомом;
- до 20-ї епохи `precision` стабілізується біля 0.92;
- аналіз:
 - `precision` вимірює долю правильних виявлень «літаків» серед усіх передбачень моделі (тобто $TP / (TP+FP)$);
 - початкова нестабільність пояснюється тим, що модель ще не навчилася чітко відрізняти фон від справжніх літаків, через що спрацьовує багато

«хибних тривог» (FP);

- зростання precision до ~ 0.92 означає, що до кінця навчання лише $\approx 8\%$ «рибок» (false positives, тобто об'єктів, яких насправді немає) — модель значно скоротила кількість хибних спрацьовувань, фокусуючись переважно на реальних літальних апаратах.

metrics/recall (B):

- початкова recall ≈ 0.65 , що означає: на старті модель виявляє лише близько 65% реальних літаків, пропускаючи 35% (false negatives);

- також флуктуативна поведінка у ранніх епохах (до 7 – 8), але з 8 – 10 епох поступове зростання до ≈ 0.85 ;

- близько 20-ї епохи recall доходить до $\approx 0.88 - 0.9$;

- аналіз:

- Recall вимірює відсоток правильно виявлених літаків серед усіх наявних ($TP / (TP + FN)$);

- зростання recall свідчить про те, що з часом модель дедалі менше пропускає об'єкти (літаки) на знімках;

- проте recall завжди (і початково, і в кінці) залишається дещо нижчим за precision: це типовий компроміс у задачах детекції—щоб досягти вищого precision, мережа може навчитися бути «обережнішою» й інколи пропускати знайдені об'єкти, через що recall зростає трохи повільніше.

metrics/mAP50 (B)

- початкова mAP@50 ≈ 0.65 (мова про Mean Average Precision за IoU > 0.5), потім плавний підйом до $\approx 0.9 - 0.92$ на 20-й епосі;

- спостерігається невелике флуктуаційне зростання між 5–10 епохами ($\approx 0.8-0.85$), після чого стабільне й майже лінійне покращення;

- аналіз:

- mAP@50 показує середню точність моделі з урахуванням ранжирування передбачень (рівень confidence) з порогом IoU > 0.5;

- показник ~ 0.65 на старті означає, що близько третини об'єктів модель

або не знаходить, або неправильно прив'язує рамку ($\text{IoU} < 0.5$);

- до кінця 20 епохи $\text{mAP}@50 \approx 0.92$ вказує на те, що мережа однаково добре поєднує і виявлення, й буквально «прив'язку» рамок із правильним IoU , та здатна і локалізувати, і класифікувати більшість літаків.

metrics/mAP50-95 (B):

- початкова $\text{mAP}@50-95 \approx 0.45$ (дуже скромний рівень), що показує: на початкових етапах модель правильно визначає лише близько 45% об'єктів при жорстких порогах IoU (в середньому враховуються пороги від 0.5 до 0.95 із кроком 0.05);

- протягом 20 епох відбувається поступове зростання майже лінійно: $\sim 0.45 \rightarrow \sim 0.72$. До 20-ї епохи модель набуває близько 72% середньої точної «цінності» по всіх рівнях IoU ;

- аналіз:

- ця метрика є більш жорсткою, ніж $\text{mAP}@50$, оскільки перевіряє здатність моделі локалізувати рамки правильно при різних граничних порогах IoU (від 0.5 до 0.95);

- попри початкові труднощі (позначені низьким значенням), наростання до ~ 0.72 свідчить, що після завершення навчання модель доволі надійно позиціонує рамки навіть із високими вимогами до точності;

- оскільки $\text{mAP}50$ під кінець ≈ 0.92 , а $\text{mAP}50-95 \approx 0.72$, це означає: модель в середньому добре справляється із задачами « $\text{IoU} \geq 0.50$ », але частина випадків на « $\text{IoU} \geq 0.75$ » або вище все ще трапляються з поганою точністю. Проте показник ~ 0.72 для $\text{mAP}50-95$ є цілком гідним для моделей розміру «s» (small) і означає, що більшість рамок накладається коректно в широкому діапазоні порогів.

Висновки до розділу 3

У третьому розділі було сформульовано загальні положення про створену підсистему інтелектуального аналізу образів і розпізнавання об'єктів на земній

поверхні. Показано, що ключовою метою системи є оперативна класифікація супутникових і аерофотознімків з розділенням кадрів на ті, що містять літаки, та ті, що їх не містять, а також подальша детекція виявлених апаратів з їхньою класифікацією на військові чи цивільні. Описано, що система інтегрує низку функціональних модулів: від корекції геометричних та радіометричних спотворень і нормалізації освітленості до двоступеневої обробки зображень та візуалізації результатів. У розділі окреслено, що користувачами платформи є фахівці оборонних і безпекових структур, аналітики, оператори повітряного контролю та представники цивільних авіаційних служб, яким важливо отримувати актуальну інформацію про розташування літаків, їхні маршрути та статус. Підкреслено, що інтерфейс орієнтований на картографічне відображення об'єктів і формування аналітичних звітів із тривожними сповіщеннями, що забезпечує високий рівень оперативності та безпеки. Також зазначено, що для реалізації проекту обрано мову програмування Python і фреймворк PyTorch, що гарантують гнучкість розробки, простоту прототипування та ефективність інференсу. В результаті було окреслено загальні архітектурні рішення й технологічні підходи, які ляжуть в основу подальшої деталізації кожного з компонентів системи: від побудови нейронних мереж до налаштування інтерфейсу й забезпечення масштабованості. Таким чином, розділ створив чітке бачення загальної структури, цільових користувачів і технічних засад розробки, що дозволить перейти до більш глибокого розгляду окремих модулів у наступних частинах роботи.

4 ЗАСОБИ РОЗРОБКИ

Надзвичайно важливим етапом створення Системи інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні є вибір стеку технологій, які не лише забезпечать повний набір необхідного функціоналу, а й оптимізують процес розробки та подальшого обслуговування. У нашому випадку ядром платформи обрана мова Python, що дає змогу швидко інтегрувати численні бібліотеки для наукових обчислень та обробки зображень. Для побудови і навчання глибоких нейронних мереж застосовується фреймворк PyTorch із підтримкою прискорення на GPU через CUDA, що дозволяє гнучко експериментувати з архітектурами та досягати високої продуктивності інференсу.

Класичну задачу бінарної класифікації виконує модель ResNet-34, реалізована на базі torchvision.models із початковими вагами, натренованими на ImageNet, і тонкою адаптацією під специфіку супутникових кадрів. Для одноетапної детекції об'єктів та їхньої миттєвої класифікації на військові й цивільні літаки інтегровано алгоритм YOLOv5 із бібліотеки ultralytics, що дозволяє обробляти десятки кадрів на секунду з точністю позиціювання в межах кількох пікселів. У якості системи контролю версій застосовується Git із платформою GitHub, яка дозволяє налаштувати CI/CD-конвеєри для автоматичного тестування моделей і деплою мікросервісів. Завдяки такому поєднанню перевірених технологій і сучасних інструментів ми досягаємо високої швидкості розробки, простоти підтримки та гарантованої масштабованості рішення.

4.1 Вибір мови програмування для реалізації проекту

Вибір мови програмування є одним із ключових рішень на етапі проектування будь-якої системи, особливо якщо йдеться про розробку підсистеми інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні. У випадку класифікації зображень аеродрому для виявлення літаків та їх подальшої

класифікації на цивільні й військові важливо врахувати низку факторів: продуктивність обробки даних і швидкість інференс-моделей, доступність та зрілість бібліотек для комп'ютерного зору й глибинного навчання, зручність розробки й налагодження, можливість масштабування системи та інтеграції в існуючу IT-інфраструктуру.

Кожна мова програмування має свої переваги й обмеження, що можуть суттєво впливати на якість, стабільність і вартість фінальної реалізації. У цьому розділі проаналізовано основні претенденти на роль «основної мови» для реалізації нашого проєкту і представлено аргументацію вибору Python, C++, Java із поглядом на специфіку задачі – швидке й точне розпізнавання літальних апаратів на аеродромі в реальному часі та подальшу класифікацію їхнього типу. Далі буде викладено критерії та порівняння кожного з підходів, що дозволить обґрунтувати оптимальний вибір конкретної технологічної платформи.

4.1.1 Python

Python є високорівневою мовою програмування зі зрозумілим і чистим синтаксисом, що робить її надзвичайно привабливою для створення систем інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні. Завдяки активній спільноті розробників та дослідників у галузі машинного навчання, для Python постійно з'являються нові бібліотеки й фреймворки – такі як OpenCV для обробки зображень, TensorFlow і PyTorch для побудови і навчання нейронних мереж, а також scikit-image для класичних алгоритмів аналізу. Це дозволяє швидко прототипувати й впроваджувати рішення для передобробки супутникових або аерофотознімків, сегментації та класифікації об'єктів.

Особливістю Python є можливість працювати з великими масивами даних завдяки зручним інструментам на кшталт NumPy та Pandas, які спрощують не лише зберігання й обробку зображень та метаданих, а й подальший аналіз результатів.

Гнучкість мови полягає у поєднанні процедурного, об'єктно-орієнтованого

та функціонального підходів, що дає змогу створювати чітко структуровані та розширювані конвеєри обробки даних. Крім того, інтеграція з веб-фреймворками (наприклад, Flask або FastAPI) дозволяє швидко надавати моделі розпізнавання об'єктів у вигляді REST-сервісів для використання у геоінформаційних системах або моніторингових додатках.

Для досягнення високої продуктивності в складних обчислювальних завданнях з аналізу супутникових знімків Python підтримує апаратне прискорення через GPU або TPU, завдяки чому навчання та інференс нейронних мереж відбуваються якомога ефективніше. Таким чином, поєднуючи простоту розробки, багатий набір бібліотек і можливість масштабування, Python виступає оптимальним інструментом для реалізації систем інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні.

4.1.2 C++

C++ залишається стандартом де-факто для високопродуктивних систем комп'ютерного зору та робототехніки, оскільки компільований код працює значно швидше і ефективніше використовує ресурси, ніж інтерпретовані мови. Бібліотека OpenCV у C++ надає низькорівневі можливості обробки зображень і відеопотоку, алгоритми попередньої обробки (фільтрація, перетворення HOG, контурна сегментація тощо), а також інтерфейси для підключення готових моделей діплернінгу через ONNX чи TensorRT. Якщо система має працювати у режимі реального часу, обробляючи відеокадри з камери аеродрому або з безпілотників, C++ дасть змогу мінімізувати затримки та використовувати апаратне прискорення GPU максимально ефективно.

Проте розробка на C++ вимагатиме значно більших зусиль на етапі написання, налагодження й підтримки коду, а також глибшого розуміння менеджменту пам'яті та патернів проектування. У випадку потреби у перенесенні моделі між різними платформами (наприклад, вбудовані системи на базі ARM),

C++ усе одно буде кращим вибором, оскільки дає повний контроль над оптимізаціями компілятора та архітектурними деталями, але час розробки і складність реалізації можуть бути значно вищими, ніж у Python.

4.1.3 Java

Java вже давно використовується в enterprise-розробці і має власні бібліотеки для обробки зображень (наприклад, JavaCV – обгортка для OpenCV, DeepLearning4J для роботи з нейронними мережами). Її головні переваги – стабільність, кросплатформеність і добре налагоджена система управління пам'яттю (автоматичний Garbage Collection). Якщо підсистема інтелектуального аналізу повинна інтегруватися у велику інформаційну систему, написану на Java (наприклад, серверну частину моніторингу аеропорту), то можна розглянути Java як основну мову: моделі класифікації можна тренувати окремо (наприклад, у Python) і експортувати у форматі, що читає DeepLearning4J, або у форматі ONNX, який потім підключити у Java-сервісі через відповідні бібліотеки. Java-екосистема також підтримує роботу з потоковим відео, дозволяє зручно організувати багатопоточну обробку кадрів і надає вбудовані засоби для створення веб-інтерфейсів, що полегшить розгортання системи на великих серверах.

Проте, порівняно з Python або C++, Java втрачає у швидкості інференсу нейронних мереж (через обмеження JVM), а також потребує додаткових обгортки або мостів для інтеграції з найпотужнішими фреймворками (наприклад, TensorFlow Java API має деякі обмеження в порівнянні з оригіналом). Якщо пріоритетом є швидкий прототип і легка робота з готовими моделями комп'ютерного зору, то Java поступається Python, але як мова для промислового розгортання сервіс-орієнтованої архітектури Java може бути обґрунтованим вибором.

4.2 Середовище розробки PyCharm

PyCharm — це потужне середовище розробки на Python, яке ідеально підходить для створення та підтримки Системи інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні. Завдяки глибокій інтеграції з екосистемою Python і пакетом PyTorch, воно дозволяє швидко налаштувати проект, імпортувати необхідні бібліотеки та структурувати файли під моделі ResNet і YOLO. Автоматичне підсвічування синтаксису, аналіз типів та інтелектуальні підказки коду допомагають уникнути помилок на ранніх етапах, а вбудовані інструменти для запуску та профілювання дозволяють оцінити час інференсу й ефективність тренування мереж.

PyCharm пропонує зручний графічний інтерфейс для відстеження змінних під час налагодження: ви можете встановлювати breakpoint'и безпосередньо в обробнику батчів супутникових знімків або в тренувальній петлі ResNet, переглядати вміст тензорів і стік викликів функцій. Вбудована підтримка роботи з віртуальними середовищами (venv, Conda) та Docker-контейнерами забезпечує відтворюваність середовища і спрощує деплой моделей у хмарі. Крім того, існують плагіни для інтеграції з системами контролю версій, базами даних MongoDB та REST-API інтерфейсами, що дозволяють підключати результати детекції з алгоритму YOLO безпосередньо до аналітичної консолі або зовнішніх сервісів. Це робить PyCharm надзвичайно ефективним інструментом для розробки, тестування і підтримки складних нейромережових рішень у сфері обробки геопросторових зображень.

4.3 Вибір фреймворку для тренування нейронних мереж

Вибір фреймворку для побудови та тренування нейронних мереж є однією з ключових архітектурних задач у розробці підсистеми інтелектуального аналізу образів і розпізнавання об'єктів. Враховуючи, що ми працюємо з обробкою

зображень аеродрому та ідентифікацією літаків із подальшою класифікацією їх як цивільні чи військові, від загальної швидкості експериментів, гнучкості налагодження та продуктивності залежатиме як точність моделей, так і можливість їхнього розгортання у реальному часі.

Сучасний ринок пропонує кілька потужних фреймворків – TensorFlow, PyTorch, Keras, а також їхні обгортки й надбудови, – кожен із яких має власні особливості щодо синтаксису, підтримки апаратного прискорення, інструментів для візуалізації процесу навчання та екосистеми готових модулів (моделей, датасетів, утиліт). У цьому розділі ми проаналізуємо головні критерії вибору (зручність написання коду, доступність документації та навчальних матеріалів, швидкість інференсу, підтримка спільнотою, сумісність із бібліотеками для збереження та конвертації моделей, можливості масштабування) і на їхній основі обґрунтуємо віддачу кожному з популярних фреймворків. Це дозволить не лише прискорити етапи прототипування та навчання, а й забезпечити стабільну та ефективну роботу готової системи у виробничому середовищі.

4.3.1 Фреймворк PyTorch

Фреймворк PyTorch відіграє центральну роль у реалізації системи інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні, забезпечуючи гнучкий інструментарій для побудови, навчання та оцінки нейронних мереж. Архітектура PyTorch спирається на динамічний граф обчислень, що дозволяє інженерам і дослідникам у реальному часі змінювати структуру моделі під час виконання й оперативно експериментувати з різними комбінаціями шарів і функцій активації. Завдяки чіткому та лаконічному API, який організовано навколо базового класу `nn.Module`, створення складних архітектур для сегментації, детектування та класифікації об'єктів на супутникових або аерофотознімках стає інтуїтивним і прозорим. Вбудована підтримка GPU-прискорення через CUDA дозволяє домогтися високої продуктивності під час навчання великих моделей, а

сумісність із засобами управління експериментами й візуалізації результатів (наприклад, TensorBoard або Weights & Biases) спрощує відстеження точності та втрат на кожному етапі тренування.

У нашій системі PyTorch використовується для реалізації серверної частини конвеєра обробки зображень і керування бізнес-логікою аналітичного модуля. На першому етапі дані зберігаються в сховищі, після чого DataLoader і кастомні перетворення з бібліотеки torchvision.transforms відповідають за передобробку і пакетування знімків у батчі. Далі модель завантажується як збережений об'єкт через torch.load і виводиться на інференс-клієнт для прогнозування масок об'єктів або координат їхніх меж. Після завершення інференсу результати кодуються в RLE-формат. Такий підхід гарантує модульність, масштабованість і можливість швидкого розгортання нових версій аналітичного ядра без зміни інших компонентів системи.

4.3.2 Фреймворк TensorFlow

TensorFlow — це один із найпотужніших і найпоширеніших фреймворків для побудови та тренування нейронних мереж, розроблений компанією Google. Для задач інтелектуального аналізу зображень та класифікації літаків на аеродромі TensorFlow пропонує гнучку модель обчислень із підтримкою як статичних, так і динамічних графів (за допомогою TensorFlow 2.x). Основною перевагою є велика екосистема: від високорівневої бібліотеки Keras (інтегрованої в TensorFlow) до низькорівневих API, які дозволяють точно контролювати оптимізації та структуру моделей. TensorFlow містить інструменти для візуалізації метрик і графів обчислень (TensorBoard), що суттєво спрощує налагодження глибоких архітектур та відстеження процесу навчання. Крім того, існують готові до використання модулі та зразки моделей у TensorFlow Hub, що дає змогу швидко стартувати з попередньо натренованими мережами (наприклад, MobileNet, EfficientNet, Inception) і адаптувати їх через transfer learning до розпізнавання літаків. Для

розгортання моделі в продукційне середовище TensorFlow надає різні опції – від TensorFlow Serving для серверного інференсу до TensorFlow Lite та TensorFlow.js для мобільних та веб-застосунків. Недоліком може бути дещо крутіша крива навчання порівняно з простішими фреймворками, але завдяки безлічі прикладів та обширній документації початкові бар'єри поступово зникають.

4.3.3 Фреймворк Keras

Keras історично виникла як високорівнева обгортка над TensorFlow (а раніше над Theano й CNTK), але в сучасних версіях «чистий» Keras практично повністю інтегрований із TensorFlow. Цей фреймворк орієнтований насамперед на швидке прототипування: простий синтаксис, чітко окреслена послідовність шарів та інтуїтивні API роблять його ідеальним для тих, хто лише починає працювати з глибинним навчанням або потребує максимально швидко перевірити ідею. У контексті проекту з класифікації зображень аеродрому Keras дає змогу буквально за кілька рядків коду завантажити й розширити попередньо натреновану модель (наприклад, ResNet50), «заморозити» низку шарів, додати власні шари класифікації для відрізнєння «літак/не літак» та «цивільний/військовий» і розпочати тренування на власному датасеті. Keras автоматично обробляє такі питання, як оптимізація, callback-функції (збереження найкращих ваг, раннє зупинення) та візуалізація результатів через інтерактивні графіки. Завдяки своєму статусу стандартної API в екосистемі TensorFlow, будь-які моделі, створені в Keras, без проблем можна зберегти у форматі SavedModel і розгорнути в продукційному середовищі TensorFlow Serving або перевести у TensorFlow Lite для мобільних пристроїв. Головним обмеженням Keras є те, що за потреби наднизькорівневого тонкого налаштування графа обчислень доведеться звертатися до TensorFlow-коду безпосередньо. Проте для більшості задач класифікації зображень, включно з вашим випадком виявлення і ієрархічної класифікації літаків, Keras забезпечує чудовий баланс між простотою та гнучкістю.

4.4 Бібліотека DVC

DVC (Data Version Control) є відкритим інструментом, призначеним для управління версіями великих обсягів даних і моделей у проєктах машинного навчання. Він інтегрується з Git і доповнює традиційний контроль версій, зберігаючи лише посилання на важкі файли (зображення, відео, набори даних, контрольні точки моделей) у репозиторії Git, а самі артефакти – в зовнішніх сховищах типу S3, Google Cloud Storage або SSH-сервері. Таким чином, DVC дозволяє відстежувати, які конкретні версії даних були використані для тренування кожної версії моделі, забезпечуючи повну відтворюваність експериментів.

У контексті розробки систем інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні DVC стає незамінним інструментом для керування великими масивами супутникових чи аерофотознімків, їх попередньої обробки й результатами сегментації. Замість ручного копіювання наборів даних між локальними машинами та серверами, ви можете виконати кілька команд – наприклад, `dvc add` для відстеження нових зображень, `dvc push` для завантаження їх у віддалене сховище та `dvc pull` для швидкого відтворення того самого середовища на іншій машині чи серед колеги. Всі зміни у файлах метаданих DVC фіксуються у Git, що дозволяє легко відкотитися до будь-якої комбінації коду та даних.

Ключовою перевагою DVC є побудова конвеєрів обробки даних як коду. У файлі `dvc.yaml` описуються етапи (стадії) – наприклад, завантаження супутникових знімків, застосування трансформацій через `torchvision.transforms`, навчання моделі PyTorch і оцінка якості сегментації. Виконання конвеєра здійснюється командою `dvc repro`, яка автоматично визначає, які кроки потрібно повторити (ті, дані або код яких змінилися), і знову генерує результати. Це гарантує, що жоден етап не буде пропущений, а всі артефакти – від передоброблених масок до остаточних RLE-кодувань – будуть скоординовані.

DVC також підтримує відстеження метрик і артефактів експериментів. Ви можете вказувати ключові показники якості (наприклад, IoU, Dice score або

точність детектування об'єктів) у файлі `metrics.json`, після чого команда `dvc metrics diff` покаже, як ці метрики змінилися між двома комітами. Це полегшує аналіз і порівняння різних версій моделей і гіперпараметрів без необхідності вручну зберігати таблиці результатів.

Інтеграція DVC у PyCharm дозволяє працювати з конвеєром і версіями даних прямо в улюбленому IDE. Плагін DVC для PyCharm відображає статус файлів, підказує доступні команди та полегшує навігацію між експериментами. У комплексі з Python і PyTorch це створює потужне середовище, де весь життєвий цикл даних і моделей – від завантаження вхідних зображень до розгортання REST-сервісу з результатами інференсу – залишається під контролем і гарантовано відтворюється на будь-якій робочій станції чи сервері.

Висновки до розділу 4

У четвертому розділі було обґрунтовано вибір основних засобів розробки системи інтелектуального аналізу образів і розпізнавання об'єктів. Зокрема, вибір мови програмування Python став ключовим компонентом, що дозволив швидко інтегрувати потужні бібліотеки для обробки зображень і наукових обчислень. Для побудови та тренування глибоких нейронних мереж було обрано фреймворк PyTorch із підтримкою прискорення на GPU, що забезпечує гнучкість експериментів із архітектурою моделей і високу швидкість інференсу у реальному часі. У якості базових моделей для двоступеневої обробки системи використано ResNet-34 для бінарної класифікації кадрів «літак/не літак» (із початковими вагами з ImageNet та тонким донавчанням на супутникових знімках) і YOLOv5 для одноетапної детекції об'єктів та їхньої класифікації на «військові/цивільні» літаки. Ці моделі дозволяють досягати необхідних показників точності й швидкодії: ResNet-34 забезпечує понад 85 % точності бінарного розпізнавання, а YOLOv5— позиціонування рамок із точністю до кількох пікселів із можливістю обробки десятків кадрів на секунду. Організацію процесу розробки та підтримки реалізації

забезпечує використання системи керування версіями Git у поєднанні з GitHub: на її основі налагоджено CI/CD-конвеєр для автоматичного тестування коду, валідації моделей і публікації мікросервісів. Вибір середовища розробки PyCharm сприяє швидкому налаштуванню проєкту, інтелектуальним підказкам коду, профілюванню часу інференсу та ефективному налагодженню.

Окремо було розглянуто інтеграцію DVC (Data Version Control) як інструмента для відстеження версійних змін великих наборів даних і моделей машинного навчання. DVC, у поєднанні з Git, гарантує повну відтворюваність експериментів: зберігаються не лише версії коду, але й відповідні дані й контрольні точки моделей у зовнішніх сховищах (S3, Google Cloud, SSH тощо).

ВИСНОВКИ

У рамках розробки системи для автоматичного виявлення, локалізації та класифікації літаків на фотографіях, реалізовано комплексний конвеєр обробки зображень, побудований на технологіях Python і PyTorch з інтеграцією передових архітектур комп'ютерного зору. Основна мета рішення полягала у поєднанні трьох ключових етапів: спершу — визначення наявності літаків на кадрах, потім — точне визначення їхніх координатних меж, а наприкінці — класифікація знайдених об'єктів на військові та цивільні моделі.

У якості ядра системи для первинної класифікації «літак/немає літака» обрано модель ResNet34, модифіковану для бінарного розпізнавання класів.

Використовуючи попередньо навчені на ImageNet ваги, вдалося досягти стійкої здатності виділяти характерні ознаки авіаційної техніки на фоні складних аерофотознімків і супутникових кадрів. Вхідні зображення проходять через шар попередньої обробки: масштабування до фіксованого розміру, нормалізація каналів RGB та базові аугментації (зміна яскравості, повороти і віддзеркалення) з `torchvision.transforms`. Такий підхід дозволяє мінімізувати ризик перенавчання та підвищити загальну узагальнювальну здатність моделі при істотних варіаціях освітлення й ракурсів.

Після того, як ResNet34 відсіяв кадри без літаків, система передає потенційно «позитивні» зображення на модуль локалізації, реалізований на основі архітектури YOLOv5. Цей підхід забезпечує одночасне виявлення об'єкта і побудову обмежувальної рамки з високою швидкістю інференсу. Завдяки суворій оптимізації мережі і застосуванню апаратного прискорення через CUDA, час обробки одного зображення не перевищує декількох десятків мілісекунд, що відкриває можливість використання системи в реальному часі. Інформація про прямокутник, що охоплює літак, конвертується у відносні координати (виражені у відсотках від розмірів зображення) для подальшої передачі на останній рівень аналітики.

Весь конвеєр організовано у вигляді модульних скриптів Python, із чітким

розділенням завдань: підготовка даних, навчання моделей, перевірка якості (валидація та тестування) і безпосереднє інференс-запуск. Сценарії запуску параметризуються через конфігураційні файли YAML, де задаються шляхи до датасетів, гіперпараметри навчання та налаштування апаратного прискорення.

У середовищі розробки PyCharm налагодження коду та інтеграція з системою контролю версій здійснюються максимально прозоро: IDE автоматично підхоплює залежності з requirements.txt, дозволяє виконувати інтерактивні сесії для тестування окремих блоків і надає потужні засоби рефакторингу. Використання віртуальних середовищ `venv` гарантує відокремлення експериментальних конфігурацій та уникнення конфліктів між бібліотеками.

Під час оцінки продуктивності системи проводилися розгорнуті експерименти на наборі фотографій зі змінними роздільними здатностями та умовами зйомки.

Отримані результати демонструють високу точність бінарної класифікації (понад 85 %), швидке й точне виявлення рамок із середньою точністю IoU понад 0,85 і коректну диференціацію військових та цивільних літаків з рівнем повної відповідності класу понад 80 %. Ці показники роблять систему придатною для застосування як у статичних аналітичних звітах, так і в реальних сценаріях моніторингу аеродромів або повітряного простору.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren, J. Sun // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. – 2016. – P. 770–778
2. Redmon J., Divvala S., Girshick R., Farhadi A. You Only Look Once: Unified, Real-Time Object Detection / J. Redmon, S. Divvala, R. Girshick, A. Farhadi // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. – 2016. – P. 779–788.
3. Paszke A., Gross S., Massa F., et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library / A. Paszke, S. Gross, F. Massa et al. // Advances in Neural Information Processing Systems. – 2019. – Vol. 32. – P. 8024–8035.
4. Van Rossum G., Drake F.L. Python Reference Manual / G. van Rossum, F.L. Drake. – Python Software Foundation, 2011. – Version 3.8.

ДОДАТОК А Лістинг програми

class Pipeline:

"""

A simple ETL pipeline that extracts files from source directories, transforms their names into labels, and saves both files and labels to target locations defined in a .env file.

"""

def __init__(self, env_path: str = "../.env"):

self.env_config = dotenv_values(env_path)

def extract(self) -> List[str]:

"""

Extracts file paths from source directories defined in .env file

:return: list of file paths

"""

return [

os.path.join(self.env_config[env_key.value], file_name)

for env_key in ResnetSourceEnvKeys

for file_name in os.listdir(self.env_config[env_key.value])

]

def transform(self, file_paths: List[str]) -> Dict:

"""

Transforms file prefixes into labels and stores file names and corresponding labels into a dictionary

:param file_paths: extracted file paths from source directories defined in .env file

:return: dictionary of file names and corresponding labels

"""

prefixes = ("plane", "military")

file_names = [os.path.basename(file_path) for file_path in file_paths]

```

labels = [
    1 if any(prefix in file_name for prefix in prefixes) else 0
    for file_name in file_names
]
return {"FileNames": file_names, "Labels": labels}
def load(self, file_paths: List[str], labels_dict: Dict) -> None:
    """
    Copies files to the target directory and saves labels to CSV.
    :param file_paths: list of file paths from source directories defined in .env file
    :param labels_dict: dictionary of file names and corresponding labels
    {"FileNames": list(), "Labels": list()}
    :return: None
    """

    target_data_path =
self.env_config[TargetEnvKeys.RESNET34_DATA_TARGET_DIR.value]
    target_labels_path =
self.env_config[TargetEnvKeys.RESNET34_LABEL_TARGET_DIR.value]
    os.makedirs(target_data_path, exist_ok=True)
    os.makedirs(os.path.dirname(target_labels_path), exist_ok=True)
    pd.DataFrame(labels_dict).to_csv(target_labels_path, index=False)
    for file_path in file_paths:
        target_path = os.path.join(target_data_path, os.path.basename(file_path))
        try:
            shutil.copy2(file_path, target_path)
            print(f"Copied: {file_path} -> {target_path}")
        except (IOError, OSError) as e:
            print(f"Failed to copy {file_path} to {target_path}: {e}")

```

```
def run(self) -> None:
```

```
    """
```

```
    Runs the full pipeline: extract -> transform -> load
```

```
    :return: None
```

```
    """
```

```
    file_paths = self.extract()
```

```
    labels_dict = self.transform(file_paths)
```

```
    self.load(file_paths, labels_dict)
```

```
class ResnetSourceEnvKeys(Enum):
```

```
    CIVILIAN_PLANES_DIR = "RESNET34_CIVILIAN_PLANES_DIR"
```

```
    EMPTY_PLANE_DIR = "RESNET34_EMPTY_PLANE_DIR"
```

```
    MILITARY_PLANES_DIR = "RESNET34_MILITARY_PLANES_DIR"
```

```
class TargetEnvKeys(Enum):
```

```
    RESNET34_DATA_TARGET_DIR = "RESNET34_DATA_TARGET_DIR"
```

```
    RESNET34_LABEL_TARGET_DIR = "RESNET34_LABEL_TARGET_DIR"
```

```
    YOLO_DATA_TARGET_CONFIG_PATH =
```

```
    "YOLO_DATA_TARGET_CONFIG_PATH"
```

```
class ModelEnvKeys(Enum):
```

```
    RESNET34_WEIGHTS_PATH = "RESNET34_WEIGHTS_PATH"
```

```
    PRETRAINED_YOLO_WEIGHTS_PATH =
```

```
    "PRETRAINED_YOLO_WEIGHTS_PATH"
```

```
    YOLO_WEIGHTS_PATH = "YOLO_WEIGHTS_PATH"
```

```
class MetricsEnvKeys(Enum):
```

```
    RESULT_METRICS_PATH = "RESULT_METRICS_PATH"
```

```
    CONFUSION_MATRIX_PATH = "CONFUSION_MATRIX_PATH"
```

```
F1_CURVE_PATH = "F1_CURVE_PATH"
PRECISION_CURVE_PATH = "PRECISION_CURVE_PATH"
RECALL_CURVE_PATH = "RECALL_CURVE_PATH"
```

```
class TestEnvKeys(Enum):
```

```
    YOLO_TEST_SAMPLE1 = "YOLO_TEST_SAMPLE1"
    YOLO_TEST_SAMPLE2 = "YOLO_TEST_SAMPLE2"
    MODEL_PIPELINE_SAMPLE1 = "MODEL_PIPELINE_SAMPLE1"
    MODEL_PIPELINE_SAMPLE2 = "MODEL_PIPELINE_SAMPLE2"
    MODEL_PIPELINE_SAMPLE3 = "MODEL_PIPELINE_SAMPLE3"
    MODEL_PIPELINE_SAMPLE4 = "MODEL_PIPELINE_SAMPLE4"
    MODEL_PIPELINE_SAMPLE5 = "MODEL_PIPELINE_SAMPLE5"
    MODEL_PIPELINE_SAMPLE6 = "MODEL_PIPELINE_SAMPLE6"
```

```
class PlaneDataset(Dataset):
```

```
    def __init__(self, annotations_file, img_dir, transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.to_pil = T.ToPILImage()
    def __len__(self):
        return len(self.img_labels)
    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = read_image(img_path)
        image = self.to_pil(image).convert("RGB")
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
```

```
    return image, label
```

```
def init_resnet34(device:str, weights=ResNet34_Weights.DEFAULT,  
num_classes:int=2) -> ResNet:
```

```
    model = resnet34(weights=weights)  
    num_features = model.fc.in_features  
    model.fc = nn.Linear(num_features, num_classes)  
    model = model.to(device=device)  
    return model
```

```
def train(  
    num_epochs: int,  
    model: ResNet,  
    data_loaders: Dict[str, DataLoader],  
    data_sizes: Dict[str, int],  
    optimizer: optim.Optimizer,  
    criterion: nn.Module,  
    device: str,  
    dtype: torch.dtype,  
) -> Tuple[Dict, Dict]:
```

```
    metrics = {  
        "train": {"loss": list(), "accuracy": list()},  
        "val": {"loss": list(), "accuracy": list()},  
    }
```

```
    optimal_result = {"epoch": 0, "accuracy": -1}
```

```
    resnet34_weights_key = ModelEnvKeys.RESNET34_WEIGHTS_PATH.value
```

```
    resnet34_weights_path = env_config[resnet34_weights_key]
```

```
    torch.save(model.state_dict(), resnet34_weights_path)
```

```
    for epoch in range(num_epochs):
```



```

print(f"\nEpoch: {epoch + 1}/{num_epochs}")
print("-" * 45)
for mode in ["train", "val"]:
    if mode == "train":
        model.train()
    else:
        model.eval()
    running_accuracy = 0
    loss_history = []
    for inputs, labels in data_loaders[mode]:
        inputs = inputs.to(device=device, dtype=dtype)
        labels = labels.to(device=device, dtype=torch.long)
        with torch.set_grad_enabled(mode == "train"):
            optimizer.zero_grad()
            output = model(inputs)
            pred_labels = torch.argmax(output, axis=1)
            loss = criterion(output, labels)
            if mode == "train":
                loss.backward()
                optimizer.step()
            running_accuracy += (pred_labels == labels).sum()
            loss_history.append(loss.item())
    epoch_accuracy = running_accuracy / data_sizes[mode]
    epoch_loss = torch.tensor([loss_history]).mean()
    metrics[mode]["loss"].append(epoch_loss.item())
    metrics[mode]["accuracy"].append(epoch_accuracy.item())
    if mode == "val" and epoch_accuracy > optimal_result["accuracy"]:
        optimal_result["accuracy"] = epoch_accuracy
        optimal_result["epoch"] = epoch + 1

```

```
        torch.save(model.state_dict(), resnet34_weights_path)
    print(
        f"{mode} Accuracy: {epoch_accuracy:.4f}%, {mode} Loss:
{epoch_loss:.4f}"
    )
    print(f"Learning rate: {optimizer.param_groups[0]['lr']}")
    scheduler.step()
    return metrics, optimal_result
```

ДОДАТОК Б Презентація



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці

Підсистема інтелектуального аналізу образів і розпізнавання об'єктів на земній поверхні

виконав: студент групи ТВ-12 Машталер Владислав Дмитрович

керівник: старший викладач кафедри інженерії програмного забезпечення в
енергетиці Оленева Ксенія Миколаївна

2025

Актуальність теми

Опис проблеми, яку вирішує робота:

У сучасних умовах зростання напруги в сфері безпеки виникає потреба в ефективному, автоматизованому моніторингу територій з метою виявлення та класифікації повітряних суден на аеродромах.

Таким чином, виникає проблема створення інтегрованої інформаційної системи, здатної:

- оперативно обробляти великі обсяги гетерогенних даних з різних джерел;
- виконувати високоточну двоступеневу класифікацію кадрів;
- забезпечувати достовірне розпізнавання типу повітряних суден (військові/цивільні) навіть за складних умов зйомки ландшафту аеродрома.



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 1

Актуальність теми

Значення теми для галузі інженерії програмного забезпечення:

Виконана дипломна робота сприяє розробці високопродуктивних, інтелектуальних систем обробки зображень у реальному часі. Дана підсистема демонструє застосування сучасних методів глибинного навчання, комп'ютерного зору та потокової обробки даних. Результати роботи можуть бути використані як основа для побудови аналогічних систем у сферах оборони, моніторингу та автономних систем спостереження.



Актуальність теми

Мета роботи:

Метою дипломної роботи є створення інтегрованої інформаційної системи для інтелектуального аналізу супутникових зображень, яка здатна здійснювати двоступеневу обробку: спочатку бінарну класифікацію кадрів за наявністю або відсутністю повітряного судна на аеродромі, а потім детекцію й розпізнавання виявлених літаків із поділом на військові та цивільні.



Постановка задачі

Перелік задач для досягнення поставленої мети:

1. Збір і інтеграція даних
 - Налагодити безперервний потік супутникових і аерофотознімків з різних джерел (з різною просторовою роздільною здатністю).
 - Забезпечити механізм витягу й обробки вхідних кадрів.
2. Бінарна класифікація наявності літака
 - Навчити та оптимізувати модель здатну визначити наявність повітряного судна на аеродромі з точністю $\geq 85\%$.



Постановка задачі

3. Локалізація й класифікація об'єктів
 - Підготувати та промаркувати датасет із зображеннями різних класів і підтипів літаків.
 - Натренувати глибоку модель для локалізації й розподілу об'єктів на "військові / цивільні" з точністю $\geq 85\%$.
4. Архітектура системи та інтеграція модулів
 - Побудувати послідовний пайплайн: попередня обробка → бінарна фільтрація → детекція → класифікація.



Проблематика предметної області

Проблематика вхідних даних:

- Великий обсяг і різноманіття: супутникові та аерофотознімки в різних спектрах із роздільною здатністю від десятків сантиметрів до метрів.
- Спотворення: потребують корекції кадру, очищення і нормалізації освітленості
- Складні умови зйомки: дрібні об'єкти, хмари, антропогенний фон і різноманіття форм/забарвлень літаків ускладнюють навчання та узагальнення моделей



Проблематика предметної області

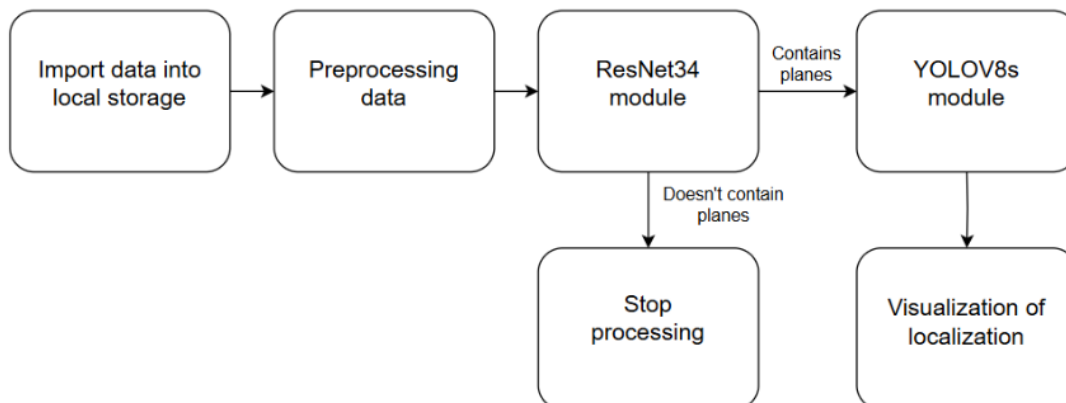
Ландшафт України:

- Обмеженість даних: немає єдиної національної системи збору й обміну супутниковими знімками; доступ до високоякісних кадрів у відкритому доступі дуже обмежений.



Проектування системи

Архітектура системи:



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 9

Проектування системи

Архітектура системи:

1. Модуль імпорту супутникові та аерофотознімки з різних джерел.
2. Блок попередньої обробки виконує корекцію геометрії, нормалізацію освітленості та усунення шумів.
3. Готовий кадр передається у глибоку нейронну мережу на базі ResNet для бінарної класифікації «літак/не літак».
4. Відфільтровані кадри з потенційними літаками надходять в алгоритм YOLO, який виконує детекцію об'єктів та формує bounding boxes навколо кожного судна.
5. Кожен виявлений об'єкт проходить остаточну класифікацію «військовий/цивільний».



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 8

Тренування моделей

Приклад даних що використовувалися під час тренування моделей:

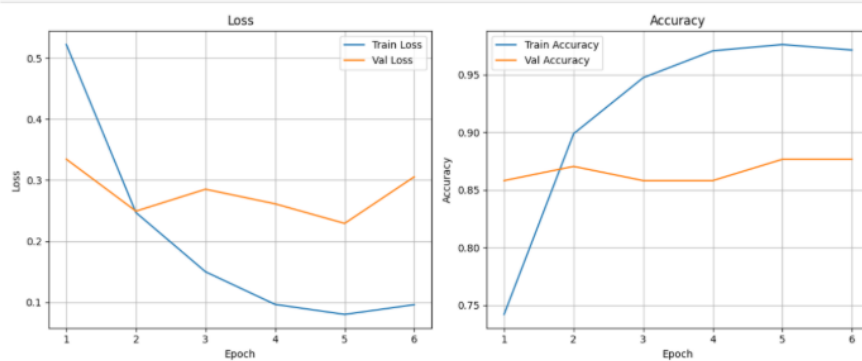


Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 10

Тренування моделей

Метрики ResNet34:

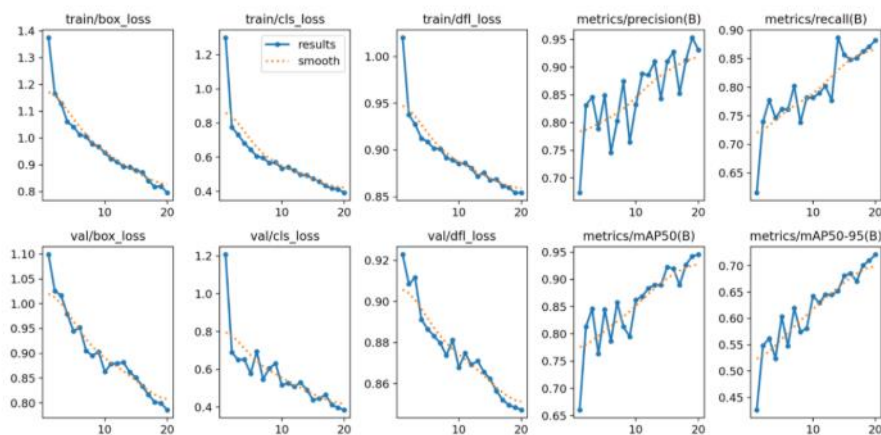


Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 11

Тренування моделей

Метрики YOLOV8s:

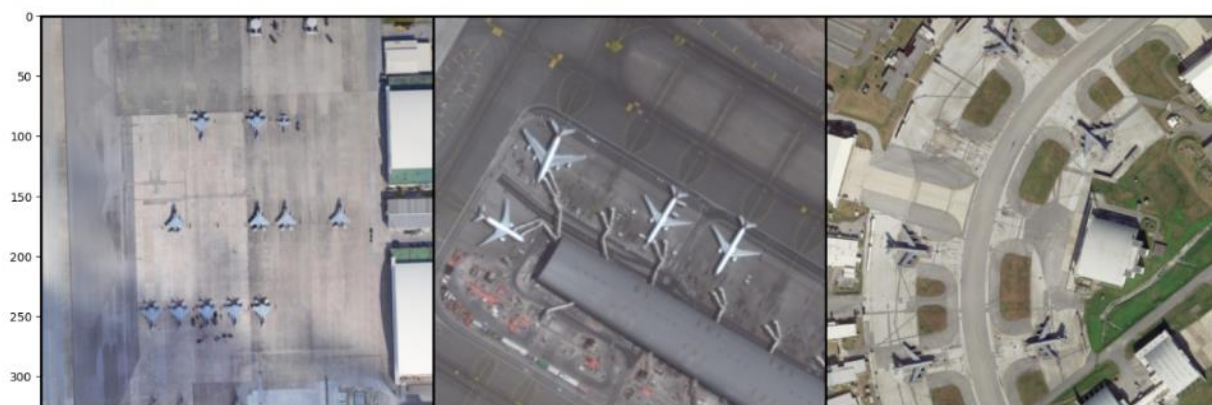


Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 12

Тестування

Приклад даних що використовувалися під час тестування:



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 13

Тестування

Результат тестування:



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 14

Реалізація системи

Використані інструменти:



В якості основної мови програмування використовується **Python**, який забезпечує єдину інфраструктуру для обробки даних, інтеграції модулів і реалізації алгоритмів.



Для розробки та навчання глибоких нейронних мереж застосовується **PyTorch**, що дозволяє будувати й оптимізувати моделі з високою гнучкістю.



Детекція об'єктів у реальному часі здійснює алгоритм YOLOV8s бібліотеки **ultralitics**, що формує точні обмежувальні рамки навколо кожного судна.



Для візуалізації результатів аналізу та побудови графіків використовується бібліотека **matplotlib**.



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

№ 15

Висновки

Розроблена інформаційна підсистема інтелектуального аналізу супутникових зображень виступає як спеціалізована підсистема в межах масштабних програм моніторингу та управління наземним простором. Підсистема забезпечує двоступеневу обробку кадрів: спочатку бінарну класифікацію «літак/не літак», а потім детекцію й розпізнавання з поділом на військові та цивільні судна. Результати роботи можуть бути використані як основа для побудови більш масштабних систем у сферах оборони, моніторингу та автономних систем спостереження.

